14

RAG: a memória que o projeto nunca teve

Cada sessão de agent começa do zero. O bug resolvido na semana passada, a decisão de arquitetura de ontem, a abordagem que falhou duas vezes — tudo evapora quando a sessão encerra. Aí você repete o contexto manualmente, gasta tokens explicando o que já explicou, ou vê o agent cometer o erro que já foi corrigido. RAG persistente é a cura para essa amnésia.

A IDEIA CENTRAL DESTE MATERIAL

RAG (Retrieval-Augmented Generation) busca conhecimento relevante numa base e o injeta no contexto antes do modelo responder. O modelo não é retreinado — recebe a informação certa na hora certa. Aplicado ao projeto: o aprendizado da sessão N fica disponível automaticamente na sessão N+1.

1 Os dois momentos: indexar e recuperar

```
INDEXAÇÃO — quando a sessão/agent encerra (em background)
Sessão termina → resumo dos aprendizados gerado
             → .md salvo em .claude/knowledge/
             → chunks de 200–300 tokens
             → embedding calculado por chunk
             → INSERT no banco vetorial (agent, categoria, data)

RECUPERAÇÃO — quando um novo prompt chega
Prompt enviado → embedding do prompt calculado
             → busca top-K chunks mais similares
             → chunks injetados no contexto
             → modelo responde já sabendo
```

É a aplicação prática dos embeddings do material 01: o "mapa de significados" agora indexa o conhecimento do *seu* projeto — e a busca é por proximidade nesse mapa, não por palavra-chave.

2 Por que chunks de 200–300 tokens

O tamanho do chunk é o parâmetro que mais afeta a qualidade. Chunk grande demais mistura muitas ideias num embedding só — a busca retorna contexto amplo e impreciso. Chunk pequeno demais é frase solta, sem contexto para o modelo usar.

Para resumos de aprendizado em markdown, 200–300 tokens por seção funcionam porque **cada seção bem escrita carrega exatamente uma ideia completa**: um bug com causa raiz, uma decisão com o motivo, um padrão com o exemplo. A forma mais simples de chunkar: cada ## vira um chunk; seção maior que 300 tokens, divide por parágrafo.

3 A stack: simples de propósito

COMPONENTE	ESCOLHA	POR QUÊ
Banco vetorial	SQLite + sqlite-vec	Embedded, sem servidor, arquivo único versionável no git. Para centenas de documentos, sobra performance. Sem Chroma, sem Qdrant, sem serviço externo.
Embeddings	@xenova/ transformers com all-MiniLM-L6-v2	Roda local em Node, ~25MB, sem API key. Suficiente para busca semântica em docs técnicos. Quer mais qualidade? Troque pela API de embeddings de um provedor — o resto da arquitetura não muda.

```
.claude/
├── knowledge/                      ← resumos .md por data/agent
│   ├── 2026-04-20-parser-rate-limit.md
│   └── 2026-04-20-backend-decisao-cache.md
├── rag.db                          ← banco vetorial
├── hooks/global/
│   ├── capture-agent-learning.sh   ← SubagentStop
│   ├── capture-session-learning.sh ← Stop
│   └── inject-context.sh           ← UserPromptSubmit
└── scripts/
    ├── summarize.ts  embed.ts  search.ts
```

Schema e categorias

```
CREATE TABLE knowledge (
  id         INTEGER PRIMARY KEY AUTOINCREMENT,
  path       TEXT NOT NULL,
  content    TEXT NOT NULL,
  embedding  BLOB NOT NULL,
  category   TEXT NOT NULL,   ← bug | decision | pattern | failure
  agent      TEXT NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
CREATE VIRTUAL TABLE knowledge_vec USING vec0(
  embedding FLOAT[384]
);
```

CATEGORIA	O QUE GUARDA
`bug`	Bug resolvido: sintoma, causa raiz, solução.
`decision`	Decisão de arquitetura: alternativas descartadas e o motivo da escolha.
`pattern`	Padrão adotado pelo time, com exemplo de uso.
`failure`	O que não funcionou e por quê — para nunca repetir.

4 Os três scripts

summarize.ts — extrai aprendizados do transcript

Recebe o transcript e usa o modelo para extrair — não um resumo geral, uma **extração focada** nas quatro categorias:

```
// essência do prompt de extração:
"Analise o transcript e extraia APENAS aprendizados concretos:

 ## Bugs resolvidos        [causa raiz + solução]
 ## Decisões de arquitetura [decisão + descartadas + motivo]
 ## Padrões adotados        [padrão + exemplo]
 ## O que não funcionou     [tentativa + por quê]

 Categoria vazia? Omita a seção. Seja conciso e específico.
 Não inclua código que não seja essencial."

// salva em .claude/knowledge/YYYY-MM-DD-[agent]-learning.md
```

embed.ts — chunka e indexa

```
// lê o .md, quebra por seção de heading, indexa:
const chunks = content.split(/^##\s/m)
                      .filter(c => c.trim().length > 50)
for (const chunk of chunks) {
  const category = detectCategory(chunk)      // pelo heading
  const output = await extractor(chunk,
    { pooling: "mean", normalize: true })     // MiniLM local
  db.prepare(`INSERT INTO knowledge
    (path, content, embedding, category, agent)
    VALUES (?, ?, ?, ?, ?)`).run(...)
}
```

search.ts — busca por similaridade

```
// embedding do prompt atual → top-3 chunks mais próximos:
const results = db.prepare(`
  SELECT k.content, k.category, k.agent,
         vec_distance_cosine(k.embedding, ?) AS distance
  FROM knowledge k
  ORDER BY distance ASC
  LIMIT 3
`).all(embedding)
// saída formatada: [categoria/agent] + conteúdo → stdout
```

5 Os três hooks: onde o RAG encontra o Harness

HOOK	PAPEL	CARACTERÍSTICA
SubagentStop	Captura o aprendizado de cada agent ao concluir a task	O ponto mais preciso: contexto pequeno e focado, sem ruído de outras conversas. Aprendizado técnico por módulo.
Stop	Captura a síntese da sessão inteira	Conhecimento estratégico: decisões de orquestração, o que ficou pendente, estado do projeto.
UserPromptSubmit	Não captura — **injeta**	Busca os chunks relevantes para o prompt atual e os entrega como contexto adicional antes do modelo responder.

```
# capture-agent-learning.sh (SubagentStop) — a essência:
INPUT=$(cat)
AGENT=$(echo "$INPUT" | jq -r '.agent_name // "unknown"')
TRANSCRIPT=$(echo "$INPUT" | jq -r '.transcript // ""')
[ -z "$TRANSCRIPT" ] && exit 0

(echo "$TRANSCRIPT" | npx ts-node .claude/scripts/summarize.ts "$AGENT" \
  | xargs -I{} npx ts-node .claude/scripts/embed.ts {} "$AGENT") &
exit 0                      ↑ background: nunca trave o fluxo

# inject-context.sh (UserPromptSubmit) — a essência:
[ ! -f ".claude/rag.db" ] && exit 0     # sem banco, sai em silêncio
CONTEXT=$(echo "$PROMPT" | npx ts-node .claude/scripts/search.ts "$AGENT")
[ -n "$CONTEXT" ] && echo "{\"additionalContext\":
  \"Conhecimento acumulado do projeto:\n\n$CONTEXT\"}"
exit 0
```

6 Tradeoffs que você precisa conhecer

TEMA	O TRADEOFF
Latência do aprendizado	Resumo e embedding rodam em background: o aprendizado da sessão N aparece na N+1, não na própria. É intencional — injetar na mesma sessão criaria loop e ruído.
Qualidade do resumo	O que entra no RAG depende do prompt de extração. RAG devolvendo ruído? Ajuste o prompt do summarize *antes* de mexer nos parâmetros de busca.
Crescimento do banco	Top-K sempre retorna os mais próximos — crescimento não degrada a qualidade, só o tempo de busca. Para anos de histórico: consolidação periódica de chunks antigos do mesmo tópico.
Versionar ou não o rag.db	No git: quem clona herda o conhecimento do time. Fora: cada dev constrói o seu. Escolha deliberada, documentada no CLAUDE.md.

7 Erros mais comuns

ERRO	CORREÇÃO
Hook síncrono travando a sessão	Summarize/embed sempre com `&` — captura é background, ponto.
search falhando sem banco	O hook verifica se `rag.db` existe e sai em silêncio se não.
Chunks gigantes	Um aprendizado por seção — um bug, uma decisão, um padrão. Nada de seções-baú.
Headings fora do padrão	Heading que não bate com as categorias vira `general` e perde priorização. Mantenha os títulos consistentes.
Injeção de contexto irrelevante	Ajuste o top-K e considere um threshold mínimo de similaridade — só injeta se a distância for menor que o corte.

> Com o tempo, esse banco vira o que o material 03 prometeu: **a memória do projeto** — o histórico de tudo que quebrou, como foi resolvido e o que nunca tentar de novo. É o componente do Harness que transforma cada sessão em investimento, não em custo.

▸ COLA PARA FIXAR

- RAG = buscar conhecimento e **injetar no contexto** na hora da consulta — sem retreinar nada.
- Dois momentos: **indexação** (ao encerrar, em background) e **recuperação** (a cada prompt).
- Chunk ideal: **200–300 tokens, uma ideia completa** — cada ## do resumo vira um chunk.
- Stack mínima que funciona: **SQLite + sqlite-vec** e **MiniLM local** — sem serviço externo, sem API key.
- Quatro categorias: **bug, decision, pattern, failure** — e o prompt de extração é o coração da qualidade.
- Três hooks: **SubagentStop** e **Stop** capturam; **UserPromptSubmit** injeta. Aprendizado de N aparece em N+1.